HybridGuard: Cross-Layer Consistency as Integrity Evidence for Android Web Environments

Anonymous Author(s)

Abstract

Web services use browser-visible environment reports to recognize clients and inform security decisions, yet reports can be modified without corresponding device changes. In cooperating Android hybrid applications, Native system measurements, WebView Host context, and App Web reports expose complementary views of an execution. Their semantic relationships can reveal report inconsistencies while accommodating legitimate environmental variation.

HybridGuard associates these observations, checks their semantic compatibility under explicit applicability conditions, and records unavailable evidence and execution failures separately. It constructs compact detectors subject to normal-alarm and definite-output constraints. The Integrated App detector combines within-view checks with Native–Web memory and timezone relations; Host geometry is evaluated separately. We release 1,509 de-identified App and App–Browser observations with documented schemas, label provenance, and validation scripts to support research on cross-view consistency.

Across 14 manipulation recipes on three emulator environments, Integrated App detects 105/126 effective modifications, versus 96/126 for separately refitted Web-only and no-memory alternatives; none raises an alarm on 252 controlled-normal positions. All nine detection differences occur in the CPU-plus-memory recipe. A fixed Native memory reference detects 18/18 tested upward modifications versus 6/18 for a global Web threshold, with no alarms on 48 normal positions. Separate Native-timezone and Host-geometry studies each retain 6/6 detections while avoiding six comparator alarms on legitimate changes. Historical records and a linked-Browser extension further expose alarm and evidence-availability costs. These findings support assessing Web-report integrity through cross-layer consistency with informative, semantically related references.

CCS Concepts

• **Security and privacy → Web application security**; *Intrusion/anomaly detection and malware mitigation.*

Keywords

Android WebView, environment report integrity, cross-layer consistency, device fingerprint manipulation, semantic references

1 Introduction

Web security decisions often depend on client reports about the execution environment. Browser-visible properties can help recognize returning clients or augment authentication [1, 7], but a plausible value can still be fabricated. Targeted browser spoofing shows that an adversary can imitate fingerprint attributes through browser modifications [9]; large-scale commercial measurements also characterize adversarial browser fingerprints [18]. Assessing report integrity therefore requires evidence beyond the plausibility of individual values.

Android hybrid applications expose complementary observations across JavaScript, the WebView host, and Android [2]. A plausible Web memory report may exceed a device-specific envelope; legitimate timezone changes may move system and Web offsets together; container resizing may legitimately change webpage geometry. These cases motivate interpreting reports against their collection context.

Prior work already uses fingerprint consistency [6, 11, 12]. We ask: **when do Native and Host references distinguish modified Web reports from legitimate environmental variation, and at what alarm and availability cost?** A reference is useful when its semantic relationship, timing, and availability permit the comparison.

HybridGuard associates Native, WebView Host, and App Web observations with an execution and evaluates semantic checks under explicit applicability conditions. It constructs compact detectors subject to constraints on normal alarms and definite-output coverage. The Integrated App detector combines within-view conditions with Native–Web memory and timezone relations. We evaluate Host geometry separately because the original App model-construction records lack the paired Host–Web snapshots this relation requires. An optional linked Browser, running in a separate execution context, adds an App–Browser timezone comparison.

On 126 controlled modifications and 252 normal positions, Integrated App detects 105 modifications versus 96 for a separately refitted Web-only detector, with no normal alarms. Relaxing the historical-normal alarm constraint adds three detections but alarms on every historical evaluation record. The Browser extension adds targeted detections while increasing unknown outcomes.

We make three contributions:

Conditional evidence semantics. An observation and decision model associates complementary Android measurements with an execution and distinguishes predicate violations, definite non-violations, unavailable evidence, and execution or association failures.

Mechanism evidence. Exploratory fixed-check comparisons examine how device-specific memory bounds, timezone co-change, and Host container context distinguish the tested report edits from legitimate variation.

Alarm and availability analysis. Refitted detectors and a linked-Browser extension quantify modification coverage, normal alarms, and definite outputs.

Code and Data Release. To support research on cross-view consistency, we release our dataset, loading and validation scripts, and documentation at https://anonymous.4open.science/r/cross-view-consistency-artifact-27D2/.

WWW '27 submission draft,
2027.

Table 1: Research focus, observation surfaces, and study profiles of related work.

Study	Research focus	Observation surfaces				Study profile		
		Native	Host	App Web	Browser	Approach	Setting	Output
FP-Scanner [11]	Spoofing-induced inconsistencies	×	×	×	✓	Inconsistency tests	Browser defenses	Alteration cues
Picasso [3]	Canvas-based device classification	×	×	×	✓	Canvas challenges	Web clients	Device-class check
FP-Inconsistent [12]	Evasive-bot fingerprint inconsistencies	×	×	×	✓	Spatial + temporal rules	Bot traffic	Bot-detection rules
BrowserFM [6]	Fingerprint dependency modeling	×	×	×	✓	Feature models	Browser configs.	Dependency queries
WVProfiler [10]	WebView fingerprint exposure	✓	×	✓	×	Framework probes	Hybrid apps	Exposure findings
Tracking Without Borders [16]	Mobile–Web tracking links	✓	×	✓	×	Static/dynamic analysis	Android apps	Tracking practices
Cross-Boundary Mobile Tracking [4]	Java-to-JavaScript data flows	✓	×	✓	×	Runtime traces	WebView apps	Data-flow paths
HybridGuard (ours)	**Cross-layer report integrity**	✓	✓[a]	✓	✓[b]	Semantic checks	Edits + benign changes	Alert / no alert / unknown

Notes. ✓ denotes an examined observation surface; × denotes a surface outside the study's reported setup. Native is the Android app/system side, Host is WebView container context, and App Web is embedded-Web observation. [a]Host geometry is evaluated separately from Integrated App. [b]Browser is an optional linked extension.

2 Background, Prior Work, and Scope

2.1 Fingerprint Consistency

Fingerprinting research examines relationships as well as individual attributes. Rendered Private identifies rendering discrepancies exploited by WebGL fingerprinting [17]. FP-Scanner detects countermeasure-induced inconsistencies, including discrepancies across JavaScript and HTTP reports [11]. Picasso compares canvas challenge responses against device-class expectations [3]. FP-Inconsistent studies spatial and temporal inconsistencies in evasive bot traffic [12]. BrowserFM models dependencies among fingerprints, system characteristics, and browser configurations [6]. Our study builds on this use of consistency as evidence.

WebView research examines both sides of the application–Web boundary. WVProfiler studies fingerprint exposure through Android WebView instrumentation [10]; Tracking Without Borders examines tracking across Java/Kotlin, JavaScript, and runtime behavior [16]; Cross-Boundary Mobile Tracking studies Java-to-JavaScript information diffusion [4]. Our focus is a current Web report's compatibility with associated Android device and container measurements, including relation applicability, missing evidence, and behavior under edits and legitimate changes. Table 1 maps these research scopes.

2.2 Surfaces and Compatibility

Let E denote the execution state and container context. An observation $x = (N(E), H(E), W(E))$ associates Native system and hardware measurements, WebView Host context, and App Web reports. The App schema has 177 catalogued fields: 84 Native, 26 Host, and 67 App Web. An optional Browser contributes 67 fields from a separate Web execution, yielding a 244-field paired schema when validly associated. Selected detectors use subsets of these fields. The associated observations are sampled separately, and all surfaces provide fallible measurements. Figure 1 summarizes the observation surfaces and their evaluation roles.

A semantic relation C_j constrains observations within a stated applicability domain. A local edit may replace $W(E)$ with $W'(E)$ while a reference stays unchanged; a legitimate change may shift related observations together. Compatibility accounts for differences in API units, precision, and abstractions. The notation E describes the execution context; checks use the collected measurements.

A violation of an applicable relation provides **conditional report-integrity evidence**: the observed values conflict with the declared constraint. The evidence establishes a relational inconsistency; identifying the altered operand, recovering true values, and determining attacker intent or authenticity require separate evidence. In evaluation, a trigger on a known effective modification counts as a detection.

2.3 Adversary and Claim Boundary

We use controlled browser overrides or emulation to edit resource, browser, locale, display, and graphics reports exposed to the Web application while a compared reference may remain unchanged. An operation counts as effective only when its effect is observed at the target surface. At inference, the detector evaluates each current record using its associated observations and the checks selected offline.

The intended setting is a cooperating Android application that collects Native and Host measurements alongside its embedded webpage. An ordinary remote website needs such integration to obtain these observations. Native and Host measurements are mutable and may share administrative control or statistical dependencies with Web reports.

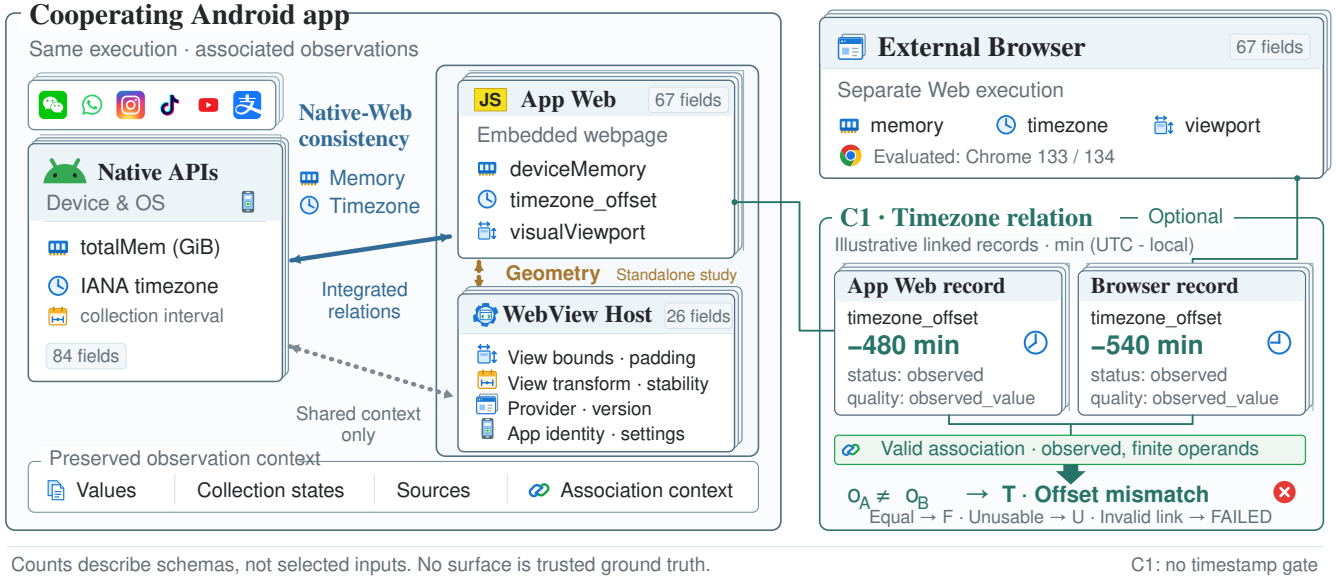


Figure 1: HybridGuard observation surfaces and evaluation roles. Solid blue denotes Native–App Web memory and timezone checks incorporated into Integrated App. Dashed ochre marks Host–Web geometry evaluated separately from Integrated App candidate selection, while dotted Native–Host indicates shared context. A validly linked external Browser enables the optional C1 timezone comparison with App Web. Field counts indicate catalogued schema attributes; detectors select subsets for evaluation.

An ALERT reports a violation of a selected relation; NO ALERT reports definite non-violation across all selected checks. Both describe the evaluated relations under available evidence. Local or coordinated edits may preserve these relations. We study controlled Web-report modifications under the stated collector and backend assumptions; collector or backend compromise, adaptive evasion, tracking, and device re-identification fall beyond this evaluation.

3 HybridGuard Design

3.1 Collection and Association

HybridGuard retains field values, collection states, association context, and provenance to bind separately sampled observations to an execution. If Browser collection is unavailable, it preserves the App record without synthesizing Browser values.

Collection context also determines applicability. For Host geometry, snapshots immediately before and after the Web snapshot establish Host stability within one collection. These are separate from the pre-intervention and post-restoration records used to evaluate the experiment. The detector uses only the current observation. Figure 2 separates this evaluation path from offline model construction.

3.2 Evidence States and Decisions

Each condition specifies its operands, source requirements, interpretation, applicability domain, and trigger. Its output is $r_j(x) \in \{\mathrm{T}, \mathrm{F}, \mathrm{U}, \mathrm{FAILED}\}$: T is an applicable, assessable trigger; F is a definite non-trigger; U indicates missing, insufficient, or inapplicable evidence; and FAILED records a relevant execution or association failure. An empty selection returns EMPTY_MODEL.

For a nonempty selection, required-input failures take precedence. Otherwise, any T produces ALERT, all F produces NO ALERT, and every remaining combination produces UNKNOWN. We record these three detector outputs as T, F, and U, respectively. A T can therefore coexist with another condition's U but cannot override FAILED. Missing unselected fields do not fail the detector.

3.3 Semantic References

Native memory envelope. For observed, finite, positive memory measurements in GiB from the same bound App record, with $m_{\mathrm{native}} \geq 0.25$, the condition is

$$r_{\mathrm{mem}}(x) = T \iff m_{\mathrm{web}} > 2^{\lceil \log_2 m_{\mathrm{native}} \rceil}.$$

Here, m_{native} is kernel-visible total memory reported by Android `ActivityManager.MemoryInfo.totalMem`, converted to GiB; m_{web} is the Web `deviceMemory` report. The historical Device Memory API draft describes approximate reporting with implementation-defined bounds [15]. Our power-of-two envelope is an engineering hypothesis that accommodates coarse reporting and permits lower Web exposure. It detects upward violations; within-envelope and coordinated modifications can remain compatible. Unusable operands yield U, and binding failures yield FAILED. The fixed Web-only comparator tests $m_{\mathrm{web}} > 8$ GiB. For example, a Web report of 8 GiB exceeds a Native-derived 4 GiB envelope but not the global threshold.

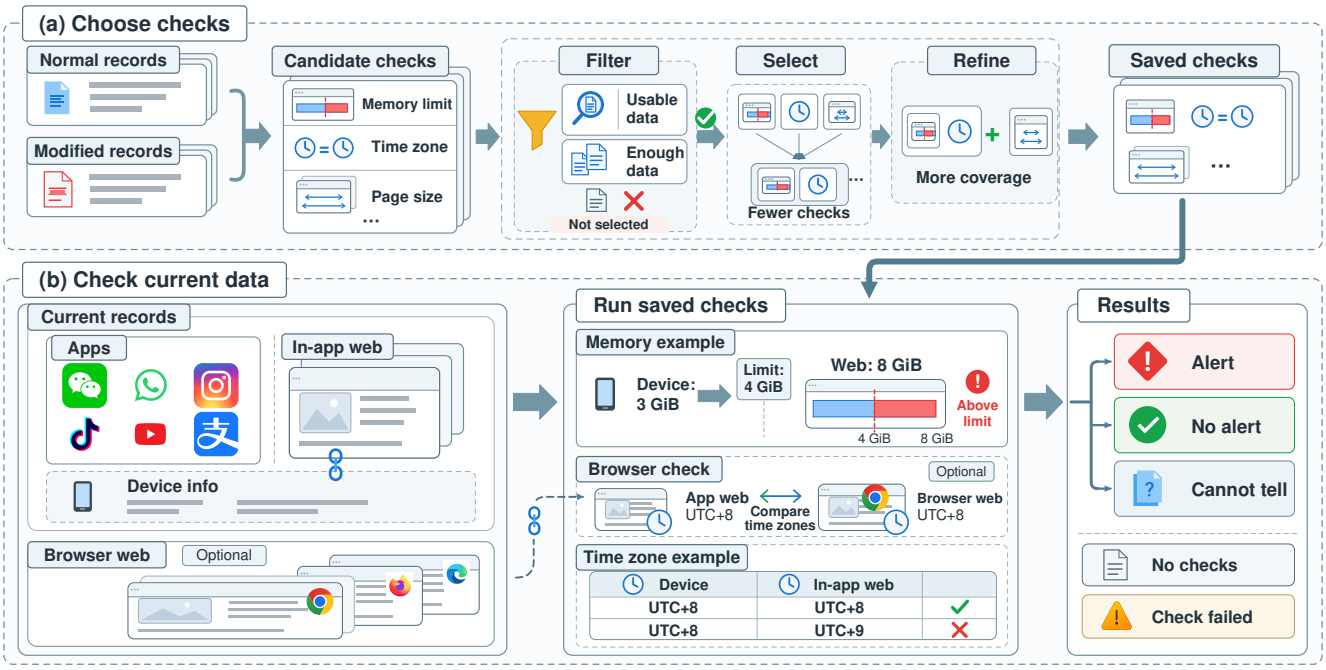


Figure 2: Constrained selection and retention freeze checks for current-record evaluation. App-only and App-plus-Browser are the evaluated configurations; Host geometry remains standalone. Outputs follow the first matching case in Section 3.2.

Native timezone reference. The Native IANA zone and collection interval determine the effective UTC offset under the fixed timezone database, 2026c. Let $o_{\text{native}}(t)$ denote local-minus-UTC minutes at the applicable collection time and o_{web} denote Web getTimezoneOffset. The trigger is $o_{\text{web}} \neq -o_{\text{native}}(t)$, accounting for the Web API's UTC-minus-local convention [5]. Supported fixed-offset identifiers can be checked without a date when consistent with the observed Native standard offset. Different zone names with equal effective offsets are compatible. Missing reliable intervals for time-dependent zones, ambiguous transitions, or unusable observations yield U; database or association failures follow the failure policy. The evaluator treats zero and -1 as valid observed offset values.

Host geometry reference (standalone). The Web interface reports visual-viewport dimensions in CSS pixels and a separate scale factor [13]. For each axis, P is the current visual-viewport CSS dimension multiplied by the current device-pixel ratio and `visualViewport.scale`; H_{view} is the measured Host View dimension minus the padding at both ends of that axis. Either width or height triggers when

$$P > H_{\text{view}} + \epsilon(P, H_{\text{view}}).$$

Here, ϵ is a pre-specified engineering tolerance; Appendix A.6 gives its exact definition and post hoc sensitivity analysis. Applicability requires snapshots from the same bound observation, Host stability across the snapshots immediately before and after the Web snapshot, finite positive dimensions, device-pixel ratio and viewport scale, an attached visible View, zero padding, and an identity View transform. Host events and before/after states determine stability independently of Host–Web agreement, allowing stable contradictions to trigger. Missing or inapplicable evidence retains U; axis outcomes follow the OR and failure rules of Section 3.2, including execution and binding errors. F records compatibility with this one-sided upper bound. Host geometry is evaluated as a standalone check outside Integrated App and its candidate pool.

3.4 Constrained Construction and Retention

The App pool contains 50 base templates and three fixed relations: Native memory, Native timezone, and same-Web viewport. The same-Web comparator flags either visual-viewport dimension exceeding the corresponding `window.innerWidth` or `window.innerHeight` by more than 1 CSS pixel, using positive, same-probe observations when visual-viewport scale is at least one. Each template defines a condition over available fields. Training-only encoding and threshold expansion instantiate 65/65/56 atoms before admission in configurations 01/02/03. Admission checks candidate polarity and assessability. Historical evaluation records do not supply threshold quantiles. A literal selects an admitted polarity; each clause contains one literal, and the detector ORs its selected clauses using Section 3.2.

For a set S of singleton clauses, the sparse stage uses

$$J(S) = \frac{1}{14} \sum_{k=1}^{14} d_k(S) - 0.005\, C(S).$$

Here k indexes the 14 manipulation recipes. For each recipe, d_k averages whole-detector detections over the three effective modifications in each training environment, then equally across the

two training environments. $C(S)$ counts clauses plus literal occurrences, assigning two complexity units to each singleton clause. With the fixed coefficient 0.005, each clause incurs a penalty of 0.01, equivalent to one percentage point of macro-averaged detection rate. The search uses greedy improving additions, the best feasible prefix, and backward pruning. This heuristic allows at most eight clauses, complexity 16, two conditions per registered family, and no opposite polarities of one atom.

Controlled-normal and historical training-normal alarm budgets are 8/168 and 31/630. Candidate assessability measures the fraction of records yielding T or F for that candidate; definite-output coverage measures the same fraction for the whole detector. Historical candidate assessability and final coverage must each reach 90%. Controlled coverage must reach 80% in each pre, modified, and post phase. Fits allow no execution failures and have a 60-second deadline.

Retention starts from each configuration's sparse solution. Semantic coverage counts distinct (semantic group, modified training record) trigger pairs using the training detection weights, with groups assigned from field semantics. Semantic quality sums predefined preference scores for positive literals, encoding observation preferences that guide retention alongside detection and normal-alarm constraints. Additions must preserve training detections, increase semantic coverage, and satisfy the alarm and coverage constraints. Same-group swaps to registered higher-quality alternatives additionally forbid new controlled-normal alarms or lost definite controlled outputs. Retention seeks additional semantic coverage or higher semantic quality without continuing to maximize J. All result cells for a rule-based App configuration use the same saved retention model.

3.5 Optional Linked Browser

Integrated App + C1 adds a timezone comparison between linked App Web and Browser records. C1 requires observed field status, observed-value quality, and finite numeric non-Boolean operands. Unequal offsets under the shared Web sign convention trigger T; missing operands yield U, and invalid association yields FAILED. C1 evaluates the paired Web offsets directly using a timestamp-independent check, whereas the Native timezone relation resolves offsets using the collection interval.

Extension selection considers no addition, C1, language condition C2, and C1+C2 on the frozen App base. Normal-alarm budgets are 31/630, 0/12, and 1/34 for historical training, pilot, and matched normals. Each normal group must retain 90% definite outputs, and historical candidate assessability must reach 90%. C2 alarms on two matched normal preference changes; C1+C2 also yields only 566/630 definite historical training outputs. We therefore retain C1 under these constraints on development-used material. The selected C1 check augments the frozen App rules, with their outputs combined under the same decision semantics.

4 Evaluation Protocol

4.1 Questions and Materials

We evaluate controlled App detection (RQ1), the contribution of semantic references (RQ2), normal alarms and evidence availability (RQ3), and the increment from a linked Browser (RQ4). Table 2 defines each study's denominator and role. Each study's reported rates summarize outcomes across its recorded evaluation positions. Because the studies have different development histories and may overlap in source records or analysis ranges, we report them separately.

Table 2: Evaluation materials and their roles. Counts denote cohort-specific record positions rather than independent devices; study groups have distinct denominators and development histories.

Subset	Δ	N	Role / boundary
Controlled			
App	126	252	Disjoint held-out positions[a]
MTC			
Discovery	—	630	Training-side constraints and tree training
Historical development	—	144	Historical evaluation with development exposure
Historically reserved	—	117	Historical evaluation; not a newly blinded cohort
Specialist			
Memory	18	48	Fixed checks; 6 no-effect operations excluded
Timezone	6	30	Includes 6 normal system changes
Geometry	6	66	Includes 6 normal layout changes; not integrated
Paired			
App–Browser	14	46	60 development-used positions: pilot 18 + matched 42

Δ = confirmed effective modifications; N = normal-context positions, including legitimate changes. MTC records are released as unlabeled. [a] Controlled App: 14 manipulation recipes × 3 repetitions × 3 environments (126 modified positions); each has a pre-intervention and post-restoration normal record (252 normal positions).

The controlled App benchmark uses API 29, 30, and 36 emulators with SwiftShader. Fourteen manipulation recipes, three repetitions, and three environments yield 126 effective modifications. Each run contributes a normal pre-intervention record and a confirmed post-restoration record, yielding 252 correlated normal positions. Model configurations 01, 02, and 03 hold out API 29, 30, and 36, respectively. Each model uses 84 modified and 168 normal training positions, then evaluates 42 modified and 84 normal held-out positions. We aggregate these disjoint held-out positions once. Prior development exposure to the recipes and environments limits claims about unseen tools and physical devices.

The mobile test-cloud (MTC) collection contains 891 records organized by manufacturer, model, and Android release, with representatives recurring across collection positions. The 630-record discovery group supplies training constraints and tree training; the historical cohorts of 144 and 117 records have prior development exposure and are reported separately. Here, "historical normal" denotes collection without a declared target manipulation, including debugging and automation context. All 891 records remain unlabeled in the public dataset because their intervention labels are unverified.

Table 3: Controlled benchmark positions: detections among nine effective modifications per original recipe; parentheses show nonzero differences from Integrated App. Only identical complete six-method state-count vectors are grouped; recipe counts retain weights. Ablations are separately refitted.

Modification group	Recipes	Integrated App	No MTC cap	Web-only	No memory	No timezone	App tree
CPU + memory	1	✓ 9	✓ 9	× 0 (−9)	× 0 (−9)	✓ 9	× 0 (−9)
Screen metrics	1	◐ 6	✓ 9 (+3)	◐ 6	◐ 6	◐ 6	× 0 (−6)
Timezone; plugins/MIME	2	✓ 9	✓ 9	✓ 9	✓ 9	✓ 9	× 0 (−9)
Shared detections†	8	✓ 9	✓ 9	✓ 9	✓ 9	✓ 9	✓ 9
Webdriver; WebGL IDs	2	× 0	× 0	× 0	× 0	× 0	× 0
Total detections	14	**105/126**	**108/126**	**96/126**	**96/126**	**105/126**	**72/126**
Detection (%)	—	83.3	85.7	76.2	76.2	83.3	57.1
Missed	—	21	18	30	30	21	54

Tested detections: ✓ all nine; ◐ $0 < d < 9$; × none. †Playwright, Puppeteer, CDP multi-field, and Stealth presets; platform-only, UA-only, UA-plus-platform, and language-list edits. All methods: 0/252 controlled-normal alarms and zero U, FAILED, and EMPTY_MODEL on these held-out positions. Saved results: RETENTION for rule models, FITTED for the tree. Three extra no-cap screen detections carry a separate normal-record cost (Table 5).

Table 4: Fixed specialist comparisons. Normal denominators include six system changes (Timezone) and six layout changes (Geometry). U and FAILED are zero. Six no-effect memory operations are excluded from 18 effective modifications and 48 normals.

Fixed check	Detected / modified	Normal alarms
Memory		
Web memory > 8 GiB	◐ 6/18	0/48
Native memory envelope	✓ 18/18	0/48
Timezone		
Absolute Web offset	✓ 6/6	△ 6/30
Native timezone reference	✓ 6/6	0/30
Geometry		
Fixed Web height	✓ 6/6	△ 6/66
Same-Web viewport relation	× 0/6	0/66
Host upper bound (standalone)	✓ 6/6	0/66

Detection on tested modifications: ✓ all; ◐ some; × none. △ nonzero normal alarms.

4.2 Compared Methods

Integrated App combines within-view and cross-view checks. Four alternatives each rebuild their sparse and retention stages: no MTC alarm cap, App Web information only, no memory, and no timezone. Removing the cap relaxes only the alarm budget; MTC assessability and coverage requirements remain. These are comparisons between refitted detectors. Web-only changes both the available information and the relations that can be constructed.

The App tree uses the same fold-specific admitted candidates, encoded as T/F/U indicators. It uses Gini splitting, depth three, and a minimum leaf size of two. Half the training weight goes to modifications, one quarter to controlled normals, and one quarter to MTC normals; modification weight is balanced across the 14 recipes. Together, these settings define a fixed-capacity decision-tree baseline over the admitted candidates. We evaluate saved fitted tree outputs and saved retention outputs for rule models. Specialist checks and Integrated App + C1 are reported separately.

4.3 Measures and Released Data

T counts as a detection on an effective modification and as an alarm on a normal-context record. We tabulate F, U, FAILED, and EMPTY_MODEL separately, retaining U in the denominators and preserving each study's record counts.

The public Cross-View Consistency Dataset v1.0 contains 1,509 observations (558 App177; 951 Paired244): 618 controlled, specialist, and linked-Browser records with labels checked against archived evidence, and 891 unlabeled MTC records. Its six no-effect memory operations are excluded from Table 2's evaluation denominators. Schemas, annotations, and explicit availability and redaction states support analysis of retained measurements. Loading and validation scripts check relationships and reproduce descriptive statistics; detector outcomes are supplied as archived annotations. Appendix A.4 details release scope and manuscript reconciliation.

5 Results

5.1 RQ1: Detection and Ablations

Integrated App detects 105/126 controlled modifications with 0/252 controlled-normal alarms (Table 3). The refitted Web-only and no-memory alternatives each detect 96/126; all nine differences occur in the CPU-plus-memory recipe. No-timezone retains 105/126 because an absolute-offset condition substitutes on this benchmark. Removing the MTC cap adds three screen detections, whose normal-data cost appears in Section 5.3. The fixed App tree detects 72/126.

All six methods miss 9/9 webdriver-only and 9/9 WebGL-identifier modifications. Historical raw WebGL unavailability and lost webdriver-false semantics limit the admission of their corresponding candidates. These misses expose gaps in the available feature representation and candidate-admission process for the tested edits. Appendix A describes the recipe grouping, and the supplemental tables retain all 84 recipe-by-method cells.

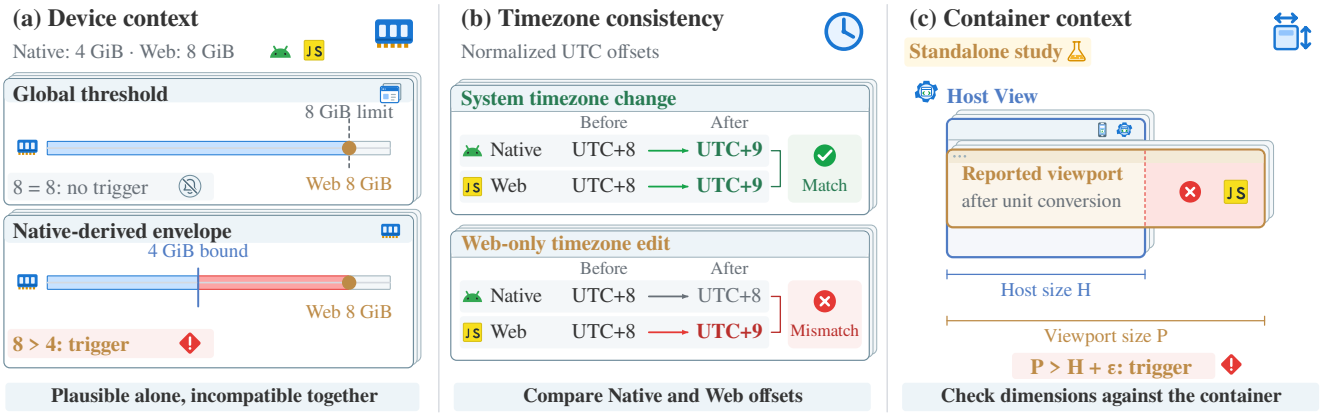


Figure 3: Illustrative mechanisms on applicable observations. (a) A Web report of 8 GiB stays within the fixed 8 GiB comparator but exceeds the envelope derived from Native memory of 4 GiB. (b) Each Native and Web row shows normalized UTC offsets before and after an illustrative change. A system timezone change moves both together, whereas a Web-only edit changes only Web. The right-hand bracket compares the two After offsets; a historical Before input is not required. (c) A converted viewport dimension P exceeding the Host bound plus tolerance triggers the standalone geometry check.

Table 5: Historical-normal decision counts by cohort. Entries report T (alarm), F (definite no alert), and U (unknown). Configurations 01 and 02 share identical per-model counts and one display row. FAILED and EMPTY_MODEL are zero throughout.

Config.	Integrated App	No cap
Historical development ($N = 144$)		
01 / 02	1 / 135 / 8	144 / 0 / 0
03	0 / 136 / 8	144 / 0 / 0
Historically reserved ($N = 117$)		
01 / 02	1 / 110 / 6	117 / 0 / 0
03	0 / 111 / 6	117 / 0 / 0

Bars show the same T / F / U proportions: dark gray / light gray / outlined white. A fully dark bar means every normal record triggered an alarm.

5.2 RQ2: What References Contribute

The specialist studies evaluate fixed checks separately from the fitted detectors to distinguish three mechanisms (Figure 3; Table 4). Their comparators use Web memory above 8 GiB, Web timezone offset above −480 minutes, and Web height above 710 CSS pixels. Appendix A.5 describes their recorded freezing and the scope of these comparisons.

Device-specific memory context. The fixed 8 GiB check also detects the nine CPU-plus-memory positions separating Integrated App from the refitted alternatives. The separate memory specialist distinguishes the checks: the Native envelope detects 18/18 modifications versus 6/18 for the global threshold, with 0/48 normal alarms for both. The twelve additional detections are upward edits to 4 and 8 GiB. Both checks detect the six edits to 16 GiB and leave the six no-effect operations targeting 2 GiB unflagged.

Legitimate timezone co-change. Both checks detect 6/6 modifications. The absolute-offset check also alarms on six normal system changes; the Native reference avoids those alarms. Thus, unchanged main-benchmark detection after removing Native timezone hides a difference in handling legitimate changes.

Host container context. The fixed Web-height check detects 6/6 modifications but alarms on 6/66 normals. The same-Web viewport relation avoids normal alarms yet misses all six modifications. The standalone Host bound achieves 6/6 detection and 0/66 alarms. In Integrated App, configurations 01/02 retain the old height check and alarm on the six normal layout changes, while configuration 03 misses the six modifications.

5.3 RQ3: Alarms and Availability

Removing the historical-normal alarm cap raises controlled detection from 105 to 108 but alarms on every record in both historical groups (Table 5). Both alternatives have 0/252 controlled-normal alarms, so the controlled benchmark alone conceals this cost. Saved uncapped models include broad conditions such as hardware concurrency greater than one and language-list length greater than one. The controlled-normal budget alone does not adequately constrain these checks on MTC records.

On the separate 630 training normals, Integrated App configurations 01/02/03 produce 6/6/0 alarms and 51 unknowns each; uncapped models alarm on all 630. We keep these training outcomes separate from evaluation. Each Integrated App configuration yields 14/261 unknowns across the two historical evaluation groups. Removing memory reduces that count to 1/261 but also loses resource detections. A tree can return a definite label when some candidate states are unknown, because its traversed path need not use those candidates.

Table 6: Paired App–Browser outcomes with development-use history. App = Integrated App; T counts detected modifications or normal alarms. U and FAILED are zero.

Record family	N	App: T	App + C1: T
App timezone	2	✓ 2	✓ 2
App language	2	× 0	× 0
Browser timezone	5	× 0	✓ 5
Browser language	5	× 0	× 0
Normal alarms	46	0	0

Modification rows: ✓ all N detected; × none detected.

5.4 RQ4: Increment from a Linked Browser

On the 60 paired App–Browser positions used during extension development (14 effective modifications and 46 normals), adding C1 raises detection from 2/14 to 7/14 (Table 6). All five additional detections arise from Browser timezone edits; App timezone detection remains 2/2, while the tested App and Browser language edits remain undetected. Both configurations yield 0/46 normal alarms. These aggregate counts hold for each of the three frozen App models and are presented once for the shared 60-position set.

C1 also raises historical unknowns from 8/144 to 11/144 in development, while leaving 6/117 in the historically reserved group. The combined count increases from 14/261 (5.4%) to 17/261 (6.5%) per model, with unchanged alarms. Asynchronous legitimate timezone changes remain an untested operating condition for C1's timestamp-independent offset comparison. The paired language tests cover App- and Browser-side report edits; the App benchmark separately evaluates a Web language-list modification recipe.

6 Discussion and Limitations

6.1 Conditional Evidence and Evasion

Native and Host observations share the execution environment, so their value as references depends on which observations an adversary can alter independently. Preserving a selected constraint may require control of only one report: a memory edit that remains within the Native-derived envelope can already evade that check. The experiments characterize responses to the tested edits; resistance to adaptive or coordinated evasion and its implications for attacker effort require further evaluation.

6.2 Generality and Development Exposure

Controlled evidence covers correlated repetitions of 14 recipes on three emulators, rather than independent physical-device samples. Prior development exposure in both historical cohorts makes the reserved cohort retrospective. Refitted ablations and the fixed-capacity tree support conclusions only for the tested methods and settings.

Development diagnostics show further limits. A relation-augmented tree fits the paired material well but does not retain that benefit across both batch-level development checks. Resource extensions improve some detections but fail the admission constraints. Appendix A and the supplemental tables report these outcomes separately from the retained detector results.

6.3 Availability, Drift, and Integration

Evidence availability shapes candidate admission and current-record decisions. Changes in APIs, WebView providers, timing, locale, or containers can affect relation applicability. The benchmark includes U outcomes and zero FAILED or EMPTY_MODEL occurrences. Deployment studies should evaluate longitudinal robustness and transfer to physical devices, estimate execution failures, rejected associations, and empty selections, and assess response policies. These policies must distinguish unresolved evidence (U) from execution or association failures (FAILED), treating both as non-definitive outcomes.

Host geometry currently applies to zero-padding, identity-transform views. Integration into model construction requires joint evaluation of detection, alarms, availability, and complexity across window modes, keyboard interactions, and physical devices. Cached-evaluation timings exclude collection and communication; end-to-end costs require measurement.

6.4 Data Collection and Privacy

Environment observations collected for integrity checking may also enable tracking [8, 14, 16]. Controlled modifications use emulators; historical MTC records come from cloud-device collection. The MTC collector requests network access and network-state observation, excluding contacts, messages, photos, location, camera, and microphone. The public release uses random record identifiers and suppresses identifying strings, high-entropy values, and precise timestamps while separating privacy suppression from collection missingness. Retained fingerprint attributes and grouping can enable cross-record linkage.

Deployments should minimize retained observations, define responses to uncertain evidence, and measure effects on authentication, account takeover, and fraud.

7 Conclusion

HybridGuard uses associated Android observations to assess the compatibility of Web environment reports. Integrated App detects 105/126 controlled modifications with no alarms on 252 normal positions; separately refitted Web-only and no-memory alternatives each detect 96/126, with all nine differences in the CPU-plus-memory recipe. Separate timezone and Host geometry studies distinguish selected edits from legitimate variation. Historical records and the Browser extension expose alarm and availability costs. The released dataset supports further analysis of retained measurements, label provenance, and missing evidence. Together, these results motivate evaluating report-integrity checks by applicability, detection, normal alarms, and definite outputs.

References

[1] Furkan Alaca and Paul C. van Oorschot. 2016. Device Fingerprinting for Augmenting Web Authentication: Classification and Analysis of Methods. In *ACSAC*. 289–301. doi:10.1145/2991079.2991091

[2] Android Developers. n.d. Build Web Apps in WebView. Living documentation. Accessed 22 October 2026. https://developer.android.com/develop/ui/views/layout/webapps/webview

[3] Elie Bursztein, Artem Malyshev, Tadek Pietraszek, and Kurt Thomas. 2016. Picasso: Lightweight Device Class Fingerprinting for Web Clients. In *SPSM*. 93–102. Author page: https://research.google/pubs/picasso-lightweight-device-class-fingerprinting-for-web-clients/. doi:10.1145/2994459.2994467

[4] Sohom Datta, Michalis Diamantaris, Ahsan Zafar, Junhua Su, Anupam Das, Jason Polakis, and Alexandros Kapravelos. 2026. Cross-Boundary Mobile Tracking: Exploring Java-to-JavaScript Information Diffusion in WebViews. In *NDSS*. doi:10.14722/ndss.2026.230910

[5] Ecma TC39. n.d. ECMAScript Language Specification. Date.prototype.getTimezoneOffset. Living specification. Accessed 22 October 2026. https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.gettimezoneoffset

[6] Maxime Huyghe, Clément Quinton, and Walter Rudametkin. 2025. BrowserFM: A Feature Model-based Approach to Browser Fingerprint Analysis. In *MADWeb*. Workshop paper: https://madweb.work/papers/2025/madweb25-huyghe.pdf. doi:10.14722/madweb.2025.23017

[7] Pierre Laperdrix, Nataliia Bielova, Benoit Baudry, and Gildas Avoine. 2020. Browser Fingerprinting: A Survey. *ACM Transactions on the Web* 14, 2, Article 8 (2020). doi:10.1145/3386040

[8] Zengrui Liu, Jimmy Dani, Yinzhi Cao, Shujiang Wu, and Nitesh Saxena. 2025. The First Early Evidence of the Use of Browser Fingerprinting for Online Tracking. In *WWW*. Association for Computing Machinery, 4980–4995. doi:10.1145/3696410.3714548

[9] Zengrui Liu, Prakash Shrestha, and Nitesh Saxena. 2022. Gummy Browsers: Targeted Browser Spoofing Against State-of-the-Art Fingerprinting Techniques. In *ACNS*. 147–169. Author preprint: https://arxiv.org/abs/2110.10129. doi:10.1007/978-3-031-09234-3_8

[10] Abhishek Tiwari, Jyoti Prakash, Alimerdan Rahimov, and Christian Hammer. 2022. Our Fingerprints Don't Fade from the Apps We Touch: Fingerprinting the Android WebView. arXiv:2208.01968. https://arxiv.org/abs/2208.01968

[11] Antoine Vastel, Pierre Laperdrix, Walter Rudametkin, and Romain Rouvoy. 2018. Fp-Scanner: The Privacy Implications of Browser Fingerprint Inconsistencies. In *USENIX Security*. 135–150. https://www.usenix.org/conference/usenixsecurity18/presentation/vastel

[12] Hari Venugopalan, Shaoor Munir, Shuaib Ahmed, Tangbaihe Wang, Samuel T. King, and Zubair Shafiq. 2025. FP-Inconsistent: Measurement and Analysis of Fingerprint Inconsistencies in Evasive Bot Traffic. In *IMC*. 134–149. Author manuscript: https://hariv.github.io/files/fpinconsistent.pdf. doi:10.1145/3730567.3732919

[13] W3C CSS Working Group. n.d. CSSOM View Module Level 1. VisualViewport interface. Editor's Draft. Accessed 22 October 2026. https://drafts.csswg.org/cssom-view-1/#the-visualviewport-interface

[14] W3C Privacy Working Group. 2025. Mitigating Browser Fingerprinting in Web Specifications. W3C Group Note. 25 September 2025. https://www.w3.org/TR/fingerprinting-guidance/

[15] W3C Web Performance Working Group. 2026. Device Memory API. W3C Discontinued Draft. 20 October 2026. Historical draft, no longer maintained. https://www.w3.org/TR/2026/DISC-device-memory-1-20261020/

[16] Nipuna Weerasekara, José Miguel Moreno, Srdjan Matic, Joel Reardon, Juan Tapiador, and Narseo Vallina-Rodríguez. 2025. Tracking Without Borders: Studying the Role of WebViews in Bridging Mobile and Web Tracking. *PoPETs* 2025, 4 (2025), 745–762. doi:10.56553/popets-2025-0155

[17] Shujiang Wu, Song Li, Yinzhi Cao, and Ningfei Wang. 2019. Rendered Private: Making GLSL Execution Uniform to Prevent WebGL-based Browser Fingerprinting. In *USENIX Security*. USENIX Association, 1645–1660. https://www.usenix.org/conference/usenixsecurity19/presentation/wu

[18] Shujiang Wu, Pengfei Sun, Yao Zhao, and Yinzhi Cao. 2023. Him of Many Faces: Characterizing Billion-scale Adversarial and Benign Browser Fingerprints on Commercial Websites. In *NDSS*. Internet Society. doi:10.14722/ndss.2023.24394

A Configuration and Reproducibility Details

A.1 Saved Model Identities

App configurations 01, 02, and 03 hold out emulator API 29, 30, and 36, respectively. APP_FULL denotes the retained B_REL_TZ models, called Integrated App in the main text. A_NO_MTC_CAP, A_APP_WEB_ONLY, A_NO_MEMORY_REL, and A_NO_TIMEZONE_REL are refitted alternatives. APP_TREE uses saved FITTED outputs; rule models use saved RETENTION outputs. PAIRED_BASE is the frozen App detector plus C1.

The candidate-state tree uses scikit-learn 1.7.2, splitter=best, random_state=20261006, no class-weight override, and a positive-class probability threshold of 0.5. Its admitted pools contain 63, 63, and 54 atoms, each encoded by three state indicators; these are smaller than the pre-admission pools. Fifteen positions in each controlled held-out set and 73/891 MTC records have an unknown candidate. A fitted path need not use that candidate, so binary predictions do not imply complete observation.

A.2 Candidate Admission and Extension Choice

Threshold expansion uses training-only quantiles 0.25, 0.5, and 0.75. Candidate support requires at least three complete true/false triplets and at least two triggered modification triplets in at least one bundle/environment, alongside normal assessability. For Browser extension, feasible choices among none, C1, C2, and C1+C2 are ranked by the equal mean detection ratio across App timezone, App language, Browser timezone, and Browser language. Ties prefer fewer additions and then a fixed set order. This ranking is part of development-side selection.

The fixed timezone comparator triggers above −480 minutes; the old Web-height comparator triggers above 710 CSS pixels. Native timezone uses effective offsets under the fixed implementation database. Geometry follows Section 3.3. C1 requires both status and quality markers, accepts finite numeric offsets including zero and minus one, and applies no timestamp gate.

A.3 Full Tables and Statistical Units

The original research archive's tables S01–S09 retain aggregate results, including adverse outcomes. Chinese source field names preserve the input schema; main-text tables use English labels. S02 contains 14 recipes × 6 methods = 84 rows. Its five count-pattern groups have sizes 1, 1, 2, 8, and 2, with nine effective modifications per recipe. Weighted sums reproduce 105, 108, 96, 96, 105, and 72 detections. App language-list edits are distinct from the paired study's language operations.

S07 reports resource-admission comparisons on 26 modified and 718 normal development positions, plus a separate replay of 261 historical normals. S0 is the frozen Integrated App + C1 baseline. M adds a Native-to-Browser memory-envelope check, W tests inequality between App-Web and Browser memory reports, and B tests Browser memory above 8 GiB. S0 detects 13/26; S0+M and S0+W detect 19/26; S0+B detects 16/26. These additions fail coverage and/or normal-alarm admission criteria, so they do not replace the retained App-plus-C1 results on 14 paired modifications.

S08 reports a four-view development diagnostic. The four-view diagnostic uses language and timezone measurements: App only, Browser only, their concatenation, and the concatenation augmented by C1/C2 states. P0 trains on the 630 discovery normals and both paired batches. P1 and P2 withhold the 18-position pilot and 42-position matched batch, respectively, while training on the discovery normals and the other batch. The relation-augmented P0 fit yields 14/14 detections and 0/46 normal alarms; under P2, the relation-augmented view yields 4/8 and 6/34. These settings are not

independent replications. S09 times cached_from_adapted execution in batches of 60 over ten rounds. Its microseconds-per-position medians and batch-amortized p95 values exclude acquisition and communication; they are neither per-request latency percentiles nor end-to-end costs.

A.4 Public Dataset and Verification Scope

The Cross-View Consistency Dataset v1.0 contains 378 controlled App, 72 memory, 36 timezone, 72 Host-geometry, 60 linked-Browser, and 891 historical MTC observations. The first four groups form 558 App177 records; the last two form 951 Paired244 records. Of the 618 experimental observations, 170 are labeled modified, 442 normal_control, and six no_effect. Labels are reconciled against saved collection and intervention evidence, separately from saved detector outputs. The six no-effect memory operations remain outside that study's 18-modification and 48-normal denominators. MTC records remain unlabeled; their 630/144/117 historical partition retains its development exposure.

Four JSONL files provide measurements, metadata, annotations, and relationships, joined by random release identifiers. Record-level links preserve 206 experiment triplets and 951 App–Browser endpoint associations. The field catalogue documents 177 App slots and 67 additional Browser slots. Original field structure is complete for 1,506 records; three MTC records each lack five original fields. Collection availability, original absence, and privacy suppression are represented separately. Recorded geometry supplements accompany 72 observations.

The release includes a dataset card, schema, field catalogue, statistics, manifest, and standalone Python tools. Running `python3 validate.py --root .` checks schema, joins, label consistency, missingness, relationships, and descriptive statistics; `python3 load_example.py --data .` demonstrates loading. Data and designated documentation are licensed under CC BY 4.0; this grant excludes the two scripts, whose software-license status is unspecified. The public artifact supports reproduction of the released data projection and its statistics. It suppresses selected identifying or decision-relevant values and omits raw logs and the full detector implementation. Detector-outcome annotations retain the original archived predictions.

Manuscript checks reconcile saved state partitions, recipe-level counts, experiment identifiers, and file hashes. Implementation tests cover decision states and binding failures.

A.5 Implementation and Comparator Provenance

The paths below identify the original research archive, which is separate from the public dataset release. It records the 50 base templates and three relations in `deliverables/app177_core_ablation_v1/CANDIDATES.json`, with dependencies in `CANDIDATE_DEPENDENCIES.md`. Final B_REL_TZ models are saved under `deliverables/timezone_relation_validation_v1/trials/B_REL_TZ__WEBGL1-LOEO-v1-{01,02,03}__RETENTION/model.json`. All three select Native–Web timezone and memory relations, MIME type count above zero, the first-language relation (`RSR-LANG-FIRST-v1`), and the registered UA/platform condition (`CAT:NW-006`). Configurations 01/02 also select Web inner height above 710 CSS pixels; configuration 03 does not. The resulting models have 6/6/5 clauses and complexity 12/12/10.

The active selector, `hybridguard_agent/research/rule_learning_v2/semantic_selection.py`, ranks feasible retention proposals by higher macro-averaged detection gain, higher semantic-coverage gain, fewer controlled-normal alarms, higher minimum phase coverage, lower complexity, higher semantic quality, and finally a structural key. The six saved B_REL_TZ fits terminate through their heuristic stopping criteria, with recorded elapsed times of 0.19–0.48 seconds. The 60-second deadline is applied to each fit and is not their recorded stopping reason.

Record-level outputs are retained in `deliverables/timezone_relation_validation_v1/predictions.jsonl.gz`, `deliverables/app177_core_ablation_v1/results/{full_predictions,predictions}.jsonl.gz`, and `deliverables/cross_endpoint_constrained_extension_v1/results/predictions.jsonl`. These saved record-level outputs support inspection beyond the aggregate tables.

Comparator provenance differs across studies. The 8 GiB condition was fixed in the memory study's `SETTINGS.json` before the 72-position collection that yielded 18 effective modifications and 48 normal positions; comparisons on the older controlled and MTC records were retrospective. The legacy 710 CSS-pixel height condition appears in the geometry study's `CONDITIONS_FROZEN.json`, fixed before the formal 72-position batch and after six engineering smoke observations. The −480-minute comparator is the legacy absolute-offset condition from the saved B_REL models; old and new model identities were frozen before the 36-position timezone collection. These are controlled mechanism studies conducted during development. The memory envelope and geometry tolerance remain engineering hypotheses within their stated domains. Appendix A.6 adds a post hoc sensitivity analysis of the Host geometry tolerance; memory-envelope sensitivity remains unexamined.

A.6 Host Geometry Tolerance and Post Hoc Sensitivity

The standalone Host geometry check uses the frozen tolerance

$$\epsilon(P, H_{\text{view}}) = 2 + 4 \cdot 2^{-23} \max(|P|, |H_{\text{view}}|, 1) \quad \text{pixels.}$$

The constant 2 allocates one pixel to each of two layout edges as a study-specific quantization assumption. The factor 2^{-23} is the float32 precision scale, and 4 is a fixed coefficient for the numerical margin. This reference scale sets the budget; the saved evaluator computes the comparison using Python `float` arithmetic. Operation-level error bounds for the rendering chain and their transfer across WebView implementations remain to be established. The specification was frozen during development after six engineering-smoke observations and before the formal v15 batch.

We perform a *post hoc sensitivity analysis* using only the 72 formal v15 observations saved in `deliverables/screen_geometry_observation_v1/predictions.jsonl.gz`. The 66 normal records satisfy `normal_basis.supported`; the six effective interventions satisfy `observable_intervention`. These are repeated observations from three emulator environments. Keeping the saved applicability, U/FAILED states, and subset labels fixed, we use each record's current Web operands and measured Host dimensions. We

Table 7: Post hoc Host geometry tolerance sensitivity on 66 normal and six effective-intervention observations.

Tolerance (pixels)	Normal alarms	Intervention detections
2	36/66	6/6
$2 + 1 \cdot 2^{-23}M$	0/66	6/6
$2 + 2 \cdot 2^{-23}M$	0/66	6/6
$2 + 4 \cdot 2^{-23}M$ (original)	0/66	6/6
$2 + 8 \cdot 2^{-23}M$	0/66	6/6
3	0/66	6/6

preserve the implementation's multiplication order: first DPR times `visualViewport.scale`, then the CSS dimension times that product. Each axis uses a strict > comparison, and axis outcomes are combined by the original OR rule. With $M = \max(|P|, |H_{\text{view}}|, 1)$, Table 7 reports the outcomes.

For each record, take the maximum of $P - H_{\text{view}}$ over width and height. Its largest normal value is approximately 2.000038 pixels; effective interventions span 475.381–664.381 pixels. Removing the relative term therefore changes decisions for normal observations near the 2-pixel boundary. Coefficients 1, 2, 4, and 8 yield identical decisions on all 72 records. We retain the original frozen coefficient 4, with the fixed 3-pixel row used only as a sensitivity comparator. This post hoc analysis leaves coefficient choice, parameter optimality, and performance against small near-boundary interventions unresolved; independent validation of a revised tolerance requires fresh observations.